

video_stream.py 파일 정리

전체 코드

```
# video_stream.py
import cv2

class VideoStream:
    def __init__(self, source=0):
        """
        source:
        - 웹캠: 0
        - RTSP: "rtsp://192.168.10.2:8554/stream"
        """
        self.source = source
        self.cap = None

    def open(self):
        self.cap = cv2.VideoCapture(self.source)

        if not self.cap.isOpened():
            print("영상 스트림 연결 실패")
            return False

        print("영상 스트림 연결 성공")
        return True

    def read(self):
        """
        프레임 1장 읽기
        return:
        - ret: 성공 여부
        - frame: 이미지 프레임
        """
        if self.cap is None:
            return False, None

        return self.cap.read()

    def get_fps(self):
        if self.cap is None:
            return 30
```

```

        fps = self.cap.get(cv2.CAP_PROP_FPS)

        if fps <= 0:
            return 30

        return fps

    def get_frame_size(self):
        if self.cap is None:
            return 640, 480

        width = int(self.cap.get(cv2.CAP_PROP_FRAME_WIDTH))
        height = int(self.cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

        return width, height

    def release(self):
        if self.cap is not None:
            self.cap.release()
            self.cap = None

```

클래스 역할

영상 스트림을 열고, 프레임을 한 장씩 읽어서 외부 코드에 전달하는 클래스

- 웹캠 연결
- RTSP 스트림 연결
- 프레임 한 장씩 읽기
- FPS 정보 반환
- 영상 크기 반환
- 사용이 끝난 카메라 자원 해제

각 함수 역할

open() 역할

- 카메라 연결
- 연결 성공 여부 반환

read() 역할

- 프레임 한장 읽기
- ret, frame 형태로 반환
 - ret : 프레임 읽기 성공 여부

- frame : 읽어온 이미지 프레임(cv이미지 배열 형태)
- open호출하지 않고 실행하면 cap이 None이기 때문에 실패 예외처리

get_fps()역할

- 현재 카메라의 FPS반환
 - 카메라 스펙의 FPS가 아닌 Opencv에서 FPS정보 가져옴
- 영상 저장, 시간 계산, 프레임 처리속도 맞추기 위해 사용
 - 영상 시간 계산
 - 30fps, 300장 프레임 = 10초 영상
- fps제대로 안받아질때 기본값 30으로 반환.(오류 방지)

get_frame_size()역할

- 현재 영상의 가로/세로크기 반환 - 해상도 가져오는 코드
- (640, 480) 이런식으로 반환

release()역할

- 사용 끝난 카메라 점유 해제, 스트림 종료등 뒷정리 함수
- 따로 반환 없음

외부 파일에서 사용하는 법

```
from video_stream import VideoStream

stream = VideoStream(source=0)
# or
stream = VideoStream(source="rtsp://192.168.10.2:8554/stream")

if not stream.open(): # open함수
    print("영상 연결 실패")
    exit()

while True:
    ret, frame = stream.read() # read함수

    if not ret:
        print("프레임 수신 실패")
        continue

    persons = tracker.track(frame) # fram YOLO에 사용
```

```
stream.release() # 연결 종료
```

클래스 흐름도

```
VideoStream 객체 생성
↓
source 저장
↓
open() 호출
↓
웹캠/RTSP 연결
↓
read() 반복 호출
↓
frame 한 장씩 반환
↓
YOLO / GUI / 영상 저장 코드에서 사용
↓
release()로 연결 해제
```